

Diseño e Implementación de Arquitecturas de Software: Cliente-Servidor, API Gateway, Capas y Tuberías

Hannah Nahid Becerril Guadarrama
Ingeniería en Tecnologías Computacionales (ITC)
Tecnológico de Monterrey

3 de junio de 2026

1. Resumen

Este proyecto presentó el desarrollo y la implementación de un sistema de recolección, indexamiento y análisis de metadatos de videos en redes sociales, estructurado bajo el paradigma de microservicios. A nivel macro, el sistema opera bajo una arquitectura Cliente-Servidor orquestada por un Application Programming Interface ([API](#)) Gateway que centraliza las peticiones de la interfaz de línea de comandos. Para resolver las distintas necesidades de procesamiento, se aplicaron dos patrones arquitectónicos a nivel microservicio: una Arquitectura por Capas (Layered Architecture) para los servicios de Crawler y Búsqueda, garantizando la separación entre presentación, negocio y datos; y un modelo de Tuberías y Filtros (Pipes and Filters) para el servicio de Indexamiento, permitiendo el procesamiento I/O secuencial de archivos multimedia. El almacenamiento se resolvió mediante una base de datos relacional SQLite, garantizando la persistencia de los metadatos y la cadena hexadecimal de los thumbnails. Se validó la resiliencia del software mediante pruebas automatizadas dinámicas conforme al estándar IEEE 29119, ejecutando más de 1,000 iteraciones. Los resultados demostraron una alta cohesión, bajo acoplamiento y un enrutamiento eficiente desde el [API](#) Gateway hacia los microservicios especializados.

2. Métodos

El desarrollo del software se ejecutó en un entorno Pop!_OS basado en Linux. Se implementó un parser central en Python (`Main.py`) que funge como la capa Servidor (API Gateway), encargado de recibir los comandos del Cliente (Command Line Interface ([CLI](#))) e instanciar las clases correspondientes sin acoplar la lógica de negocio.

2.1. Arquitectura Cliente-Servidor y API Gateway

El paradigma Cliente-Servidor rige la interacción de todo el sistema. El usuario interactúa a través de comandos en la terminal (Cliente), los cuales son interceptados por el patrón API Gateway (`Main.py`). Este orquestador abstrae la complejidad del backend y enruta las peticiones de acuerdo con el comando solicitado (`Crawl`, `Retrieve`, `List`, `Dashboard`, `Preguntar`, `Buscar`) hacia el microservicio correspondiente, estandarizando las respuestas en formato JavaScript Object Notation ([JSON](#)) o mediante renderizado de interfaces gráficas.

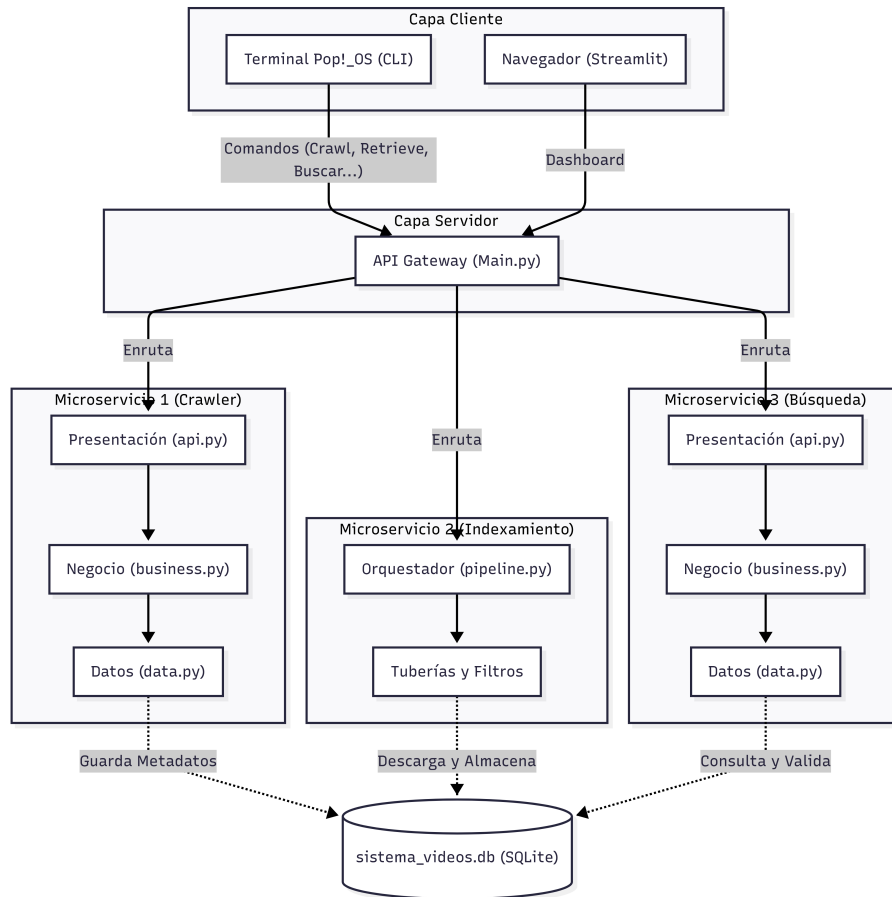


Figura 1: Diagrama UML de la arquitectura Cliente-Servidor y API Gateway.

2.2. Arquitectura por Capas (Microservicios 1 y 3)

Para el Microservicio 1 (Crawler) y el Microservicio 3 (Búsqueda), se implementó una Arquitectura por Capas estricta. El diseño físico y lógico se dividió en tres niveles para fomentar la cohesión:

1. **Capa de Presentación / API (api.py):** Actúa como el controlador que recibe la petición del API Gateway. No contiene lógica compleja; su única responsabilidad es invocar a la capa inferior e imprimir el estatus de éxito.
2. **Capa de Lógica de Negocio (business.py):** Contiene las reglas del dominio. En el Crawler, transforma los datos a formato diccionario, genera estampas de tiempo y decodifica el thumbnail a formato `hex64`. En el de Búsqueda, aplica el motor probabilístico basado en matrices de adyacencia para validar rangos numéricos.
3. **Capa de Acceso a Datos (data.py):** Abstrae la persistencia. Se encarga de las conexiones directas a SQLite y la ejecución de las sentencias Structured Query Language (SQL) como `INSERT OR REPLACE`.

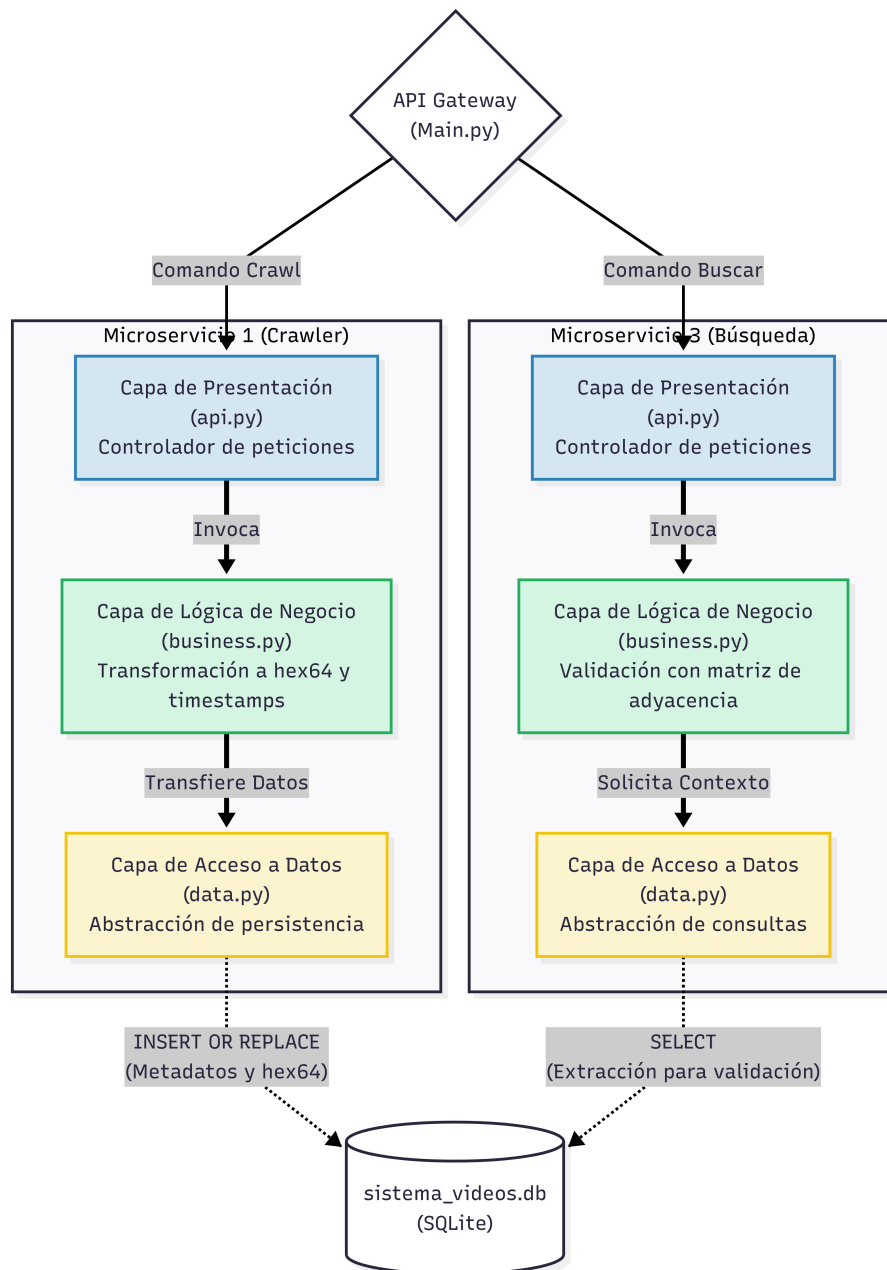


Figura 2: Diagrama UML de la Arquitectura por Capas implementada en MS1 y MS3.

2.3. Arquitectura Tuberías y Filtros (Microservicio 2)

El Microservicio 2 (Indexamiento) se construyó utilizando el patrón de Tuberías y Filtros para modularizar la ingesta y transformación de archivos multimedia. Se sustenta en una clase interfaz abstracta `Filtro` que define el contrato `procesar(data: str) ->str`. La clase orquestadora `PipelineIndexamiento` inicializa un arreglo secuencial donde la salida de un filtro se inyecta como entrada del siguiente.

Los componentes específicos operan de la siguiente manera:

1. **EntradaNormalizacionFiltro:** Analiza la cadena de entrada; si detecta una Uniform Resource Locator ([URL](#)), invoca la librería `yt-dlp` con `format: 'worst'` para descargar físicamente el video en `./Downloads/`. Posteriormente, aplica una pausa obligatoria (`time.sleep()`) antes de emitir la ruta absoluta.

2. **ProcesamientoVideoFiltro:** Recibe la ruta local del archivo multimedia y simula la transformación invocando comandos de escalado a 256 píxeles de altura mediante FFmpeg.
3. **AlmacenamientoFiltro:** Establece conexión con `sistema_videos.db` y mediante la directiva `SQL INSERT OR IGNORE`, extrae los metadatos generados a lo largo de la tubería, consolidándolos en el esquema relacional.

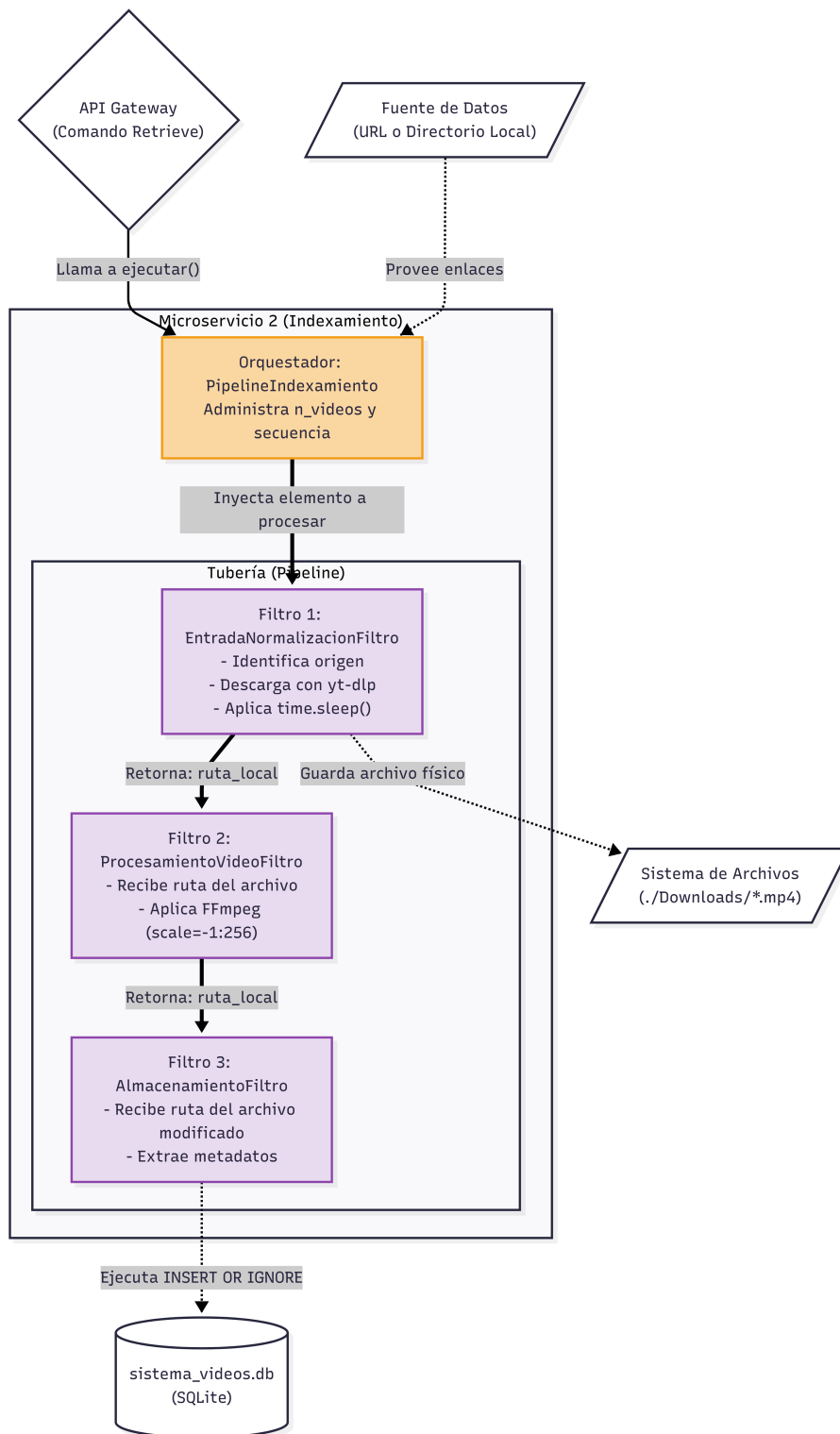


Figura 3: Diagrama de componentes UML de la arquitectura Tuberías y Filtros en MS2.

3. Resultados

A continuación, se detalla la estructuración de los datos y la evidencia de ejecución del sistema, separados por componente arquitectónico y validando el correcto funcionamiento del parser principal.

3.1. Estructuración de la Data y Base de Datos

Los metadatos se consolidaron en una única tabla relacional estructurada de la siguiente manera:

Campo	Descripción Técnica
id	Llave primaria autoincremental (INTEGER).
url	Enlace o ruta local del archivo (TEXT UNIQUE).
reaccion_social	Desglosado en vistas y likes (INTEGER).
thumbnail_hex64	Cadena decodificada de Base64 a hexadecimal (TEXT).
timestamp	Estampa de tiempo de captura formato de fecha y hora.
duration	Duración del video.
comments	Comentarios del video.
categoria	Clasificación del contenido (TEXT).

Cuadro 1: Esquema de almacenamiento de SQLite.

3.2. Llamados desde la API (Backend)

Las siguientes terminales demuestran la operación de los comandos ejecutados a través del parser central `Main.py`, evidenciando la separación de los microservicios.

```
(Proyecto_Fleyva) hannahbeceril@pop-os:~/Documents/proyecto_final$ python Main.py Crawl -URL https://www.youtube.com/@lacomer1/videos
[Crawler] Iniciando extracción de datos para: https://www.youtube.com/@lacomer1/videos...
[Crawler] ✓ Metadatos guardados correctamente en la base de datos.
```

Figura 4: Ejecución en terminal del Microservicio 1 (Arquitectura por Capas). Comando utilizado: `python Main.py Crawl -URL https://www.youtube.com/@lacomer1/videos`

```
(Proyecto_Fleyva) hannahbeceril@pop-os:~/Documents/proyecto_final$ python Main.py Retrieve -URL https://www.youtube.com/@lacomer1/videos -N 30 -Sleep 10 -Path ./Downloads/[Pipeline]
Obteniendo lista de videos del canal: https://www.youtube.com/@lacomer1/videos

[Filtro Entrada] Inicializando descarga de URL: https://www.youtube.com/watch?v=yitgXqinpKQ...
WARNING: [youtube] No supported JavaScript runtime could be found. Only deno is enabled by default; to use another runtime add --js-runtimes RUNTIME[:PATH] to your command/config.
g. YouTube extraction without a JS runtime has been deprecated, and some formats may be missing. See https://github.com/yt-dlp/yt-dlp/wiki/EJS for details on installing one
[Filtro Entrada] ✓ Video descargado en: ./Downloads/yitgXqinpKQ.mp4
[Filtro Entrada] Aplicando Sleep de 10 segundos...
[Filtro Procesamiento] Escalando video a 256h pixeles: ./Downloads/yitgXqinpKQ.mp4
[Filtro Almacenamiento] Extrayendo metadatos e indexando en DB: ./Downloads/yitgXqinpKQ.mp4

[Filtro Entrada] Inicializando descarga de URL: https://www.youtube.com/watch?v=QQ4g5XXji6I...
WARNING: [youtube] No supported JavaScript runtime could be found. Only deno is enabled by default; to use another runtime add --js-runtimes RUNTIME[:PATH] to your command/config.
g. YouTube extraction without a JS runtime has been deprecated, and some formats may be missing. See https://github.com/yt-dlp/yt-dlp/wiki/EJS for details on installing one
[Filtro Entrada] ✓ Video descargado en: ./Downloads/QQ4g5XXji6I.mp4
[Filtro Entrada] Aplicando Sleep de 10 segundos...
[Filtro Procesamiento] Escalando video a 256h pixeles: ./Downloads/QQ4g5XXji6I.mp4
[Filtro Almacenamiento] Extrayendo metadatos e indexando en DB: ./Downloads/QQ4g5XXji6I.mp4

[Filtro Entrada] Inicializando descarga de URL: https://www.youtube.com/watch?v=V3CFk7FBvkv...
WARNING: [youtube] No supported JavaScript runtime could be found. Only deno is enabled by default; to use another runtime add --js-runtimes RUNTIME[:PATH] to your command/config.
g. YouTube extraction without a JS runtime has been deprecated, and some formats may be missing. See https://github.com/yt-dlp/yt-dlp/wiki/EJS for details on installing one
[Filtro Entrada] ✓ Video descargado en: ./Downloads/V3CFk7FBvkv.mp4
[Filtro Entrada] Aplicando Sleep de 10 segundos...
[Filtro Procesamiento] Escalando video a 256h pixeles: ./Downloads/V3CFk7FBvkv.mp4
[Filtro Almacenamiento] Extrayendo metadatos e indexando en DB: ./Downloads/V3CFk7FBvkv.mp4

[Filtro Entrada] Inicializando descarga de URL: https://www.youtube.com/watch?v=uYig3j7fIM...
WARNING: [youtube] No supported JavaScript runtime could be found. Only deno is enabled by default; to use another runtime add --js-runtimes RUNTIME[:PATH] to your command/config.
g. YouTube extraction without a JS runtime has been deprecated, and some formats may be missing. See https://github.com/yt-dlp/yt-dlp/wiki/EJS for details on installing one
[Filtro Entrada] ✓ Video descargado en: ./Downloads/uYig3j7fIM.mp4
[Filtro Entrada] Aplicando Sleep de 10 segundos...
```

Figura 5: Ejecución en terminal del Microservicio 2 (Tuberías y Filtros). Comando utilizado: `python Main.py Retrieve -URL https://www.youtube.com/@lacomer1/videos -N 30 -Sleep 10 -Path ./Downloads/`

```
(Proyecto_Fleyva) hannahbecerill@pop-os:~/Documents/proyecto_final$ python Main.py List -URL https://www.youtube.com/@lacomere1/videos
[List] Consultando DB en la ruta absoluta:
/home/hannahbecerill/Documents/proyecto_final/sistema_videos.db

./Downloads/182zBJ958s.mp4
./Downloads/1A_34p1F_5U.mp4
./Downloads/E4odqQL18o.mp4
./Downloads/H3d8os194E1.mp4
./Downloads/L24USRvVhPg.mp4
./Downloads/MMAUnZq7YU.mp4
./Downloads/QQ4g5XXji6I.mp4
./Downloads/V3CFK7FBvK.mp4
./Downloads/ZELz40B4qU4.mp4
./Downloads/_RL5CJuxaKo.mp4
./Downloads/_1W-aNPakm8.mp4
./Downloads/_ewSgoJShCA.mp4
./Downloads/dKPjFTxvK8Q.mp4
./Downloads/dQ5XvZ6Q90w.mp4
./Downloads/eemwonheeNQ.mp4
./Downloads/fm-GSKpztw.mp4
./Downloads/goXVocQNsKE.mp4
./Downloads/guTmkuE4N04.mp4
./Downloads/12zVW5x1bp0.mp4
./Downloads/1GAE1NMq6M.mp4
./Downloads/j7MSQ19j7yc.mp4
./Downloads/jXxNm43w6A.mp4
./Downloads/jjoJHNU1g6Y.mp4
./Downloads/o-NuATK82H0.mp4
./Downloads/q-qqJJaPzwY.mp4
./Downloads/uPvQ8BY6yno.mp4
```

Figura 6: Ejecución en terminal verificando la persistencia en DB. Comando utilizado: `python Main.py List -URL https://www.youtube.com/@lacomere1/videos`

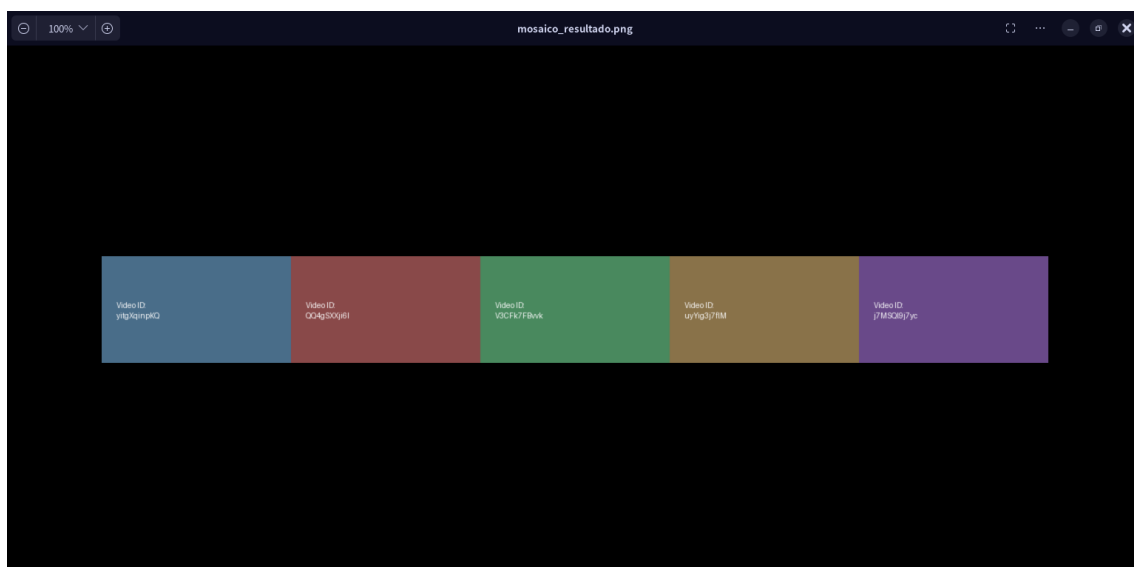


Figura 7: Mosaico 1:5 generado automáticamente tras la consulta al Large Language Model (LLM). Comando utilizado: `python Main.py Preguntar -Q "¿En qué video hablan del Cobia smart phone?"`

3.3. Operación del Dashboard (Frontend)

El comando `Dashboard` detonó un subproceso interno empleando la librería `subprocess` para levantar el servidor interactivo de Streamlit, consumiendo directamente el contexto almacenado en la capa de datos.

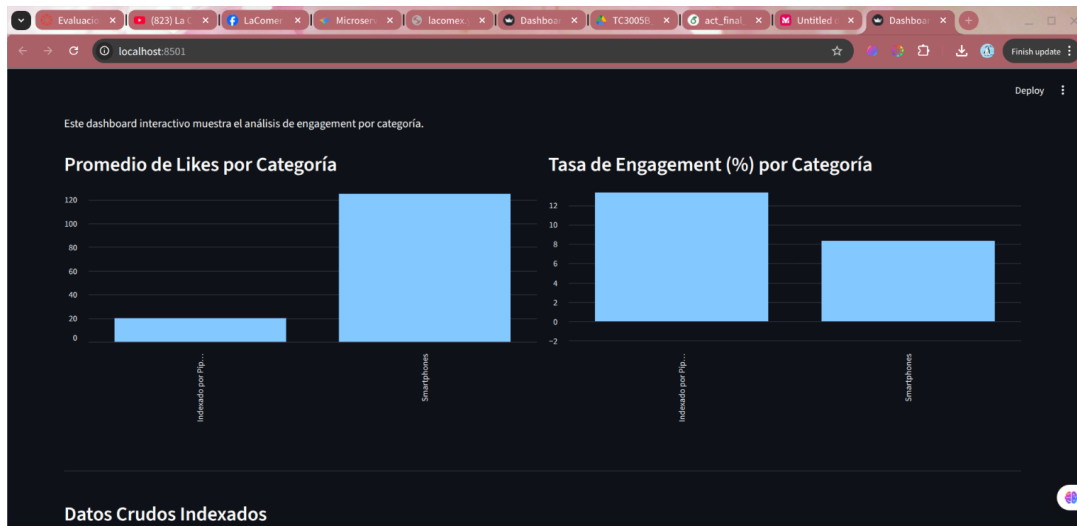


Figura 8: Visualización del dashboard de reacciones sociales renderizado vía Streamlit. Comando utilizado: `python Main.py Dashboard`

Datos Crudos Indexados

	url	likes	vistas	categoria	engagement_rate
0	./yItgXqnpKQ.mp4	20	150	Indexado por Pipeline	13.3333
1	./Q4g5XjjeI.mp4	20	150	Indexado por Pipeline	13.3333
2	./V3CFk7FBvK.mp4	20	150	Indexado por Pipeline	13.3333
3	./uyYig3j7fM.mp4	20	150	Indexado por Pipeline	13.3333
4	./j7MSQl9j7yc.mp4	20	150	Indexado por Pipeline	13.3333
5	./o-NuJaTK82H0.mp4	20	150	Indexado por Pipeline	13.3333
6	./eemwonheeNQ.mp4	20	150	Indexado por Pipeline	13.3333
7	./l82rBJ905Bz.mp4	20	150	Indexado por Pipeline	13.3333
8	./jKwMm43w6A.mp4	20	150	Indexado por Pipeline	13.3333
9	./dQSkvZ6Q90w.mp4	20	150	Indexado por Pipeline	13.3333

Figura 9: Visualización del dashboard de reacciones sociales renderizado vía Streamlit. Comando utilizado: `python Main.py Dashboard`

4. Conclusiones

La combinación de paradigmas bajo un macro-modelo Cliente-Servidor demostró ser la estrategia más eficaz para este desarrollo. El uso de un [API Gateway](#) eliminó la dispersión del código al proveer un punto de entrada unificado y transparente para el usuario. Por debajo de este orquestador, asignar Arquitecturas por Capas a los servicios de Crawling y Búsqueda permitió encapsular la lógica de negocio y las conexiones relacionales de manera segura, facilitando el mantenimiento y las pruebas automatizadas. Simultáneamente, abordar el Indexamiento de videos con un enfoque de Tuberías y Filtros probó ser indispensable para manejar operaciones costosas de I/O de red, logrando que cada fase de la descarga y manipulación multimedia operara sin sobrecargar la memoria. Finalmente, la integración del script bash cumpliendo con el estándar IEEE 29119 validó la tolerancia a fallos del motor probabilístico, entregando un producto de software integral, altamente modular y listo para su implantación en entornos de producción.

5. Resumen de Comandos

A continuación se presenta el catálogo completo de comandos implementados en la interfaz de línea de comandos ([CLI](#)) a través del [API](#) Gateway:

- **Crawl (Microservicio 1):** `python Main.py Crawl -URL <url_canal>`
- **Retrieve (Microservicio 2):** `python Main.py Retrieve -URL <url_canal>-N <num_videos>-Sleep <segundos>-Path <ruta_local>`
- **List (Microservicio 2):** `python Main.py List -URL <url_canal>`
- **Preguntar (Microservicio 2):** `python Main.py Preguntar -Q «pregunta_texto>`
- **Dashboard (Microservicio 2):** `python Main.py Dashboard`
- **Buscar (Microservicio 3):** `python Main.py Buscar -Likes » <cantidad>`

Glosario

API Application Programming Interface

CLI Command Line Interface

REST Representational State Transfer

JSON JavaScript Object Notation

LLM Large Language Model

SQL Structured Query Language

UML Unified Modeling Language

URL Uniform Resource Locator

IEEE Institute of Electrical and Electronics Engineers

Referencias

1. IEEE Computer Society. (2013). *IEEE Standard for Software and Systems Engineering—Software Testing* (IEEE Std 29119).
2. Richards, M. (2015). *Microservices vs. Service-Oriented Architecture*. O'Reilly Media.
3. Hohpe, G., & Woolf, B. (2003). *Enterprise Integration Patterns*. Addison-Wesley.

A. Apéndice: Prueba Automática Dinámica (IEEE 29119)

Script en Bash desarrollado para someter al Microservicio 3 a una prueba de carga secuencial, verificando la detección de anomalías probabilísticas mediante 1,000 iteraciones automáticas. Los resultados se inyectan dinámicamente en el archivo de salida `resultados_pruebas.out`.

```
1 #!/bin/bash
2 OUTPUT_FILE="resultados_pruebas.out"
3
4 echo "Iniciando suite de pruebas de estres (1000 iteraciones)..." >
   $OUTPUT_FILE
5 echo "-----" >> $OUTPUT_FILE
6
7 for i in {1..1000}
8 do
9     RANDOM_LIKES=$((1 + $RANDOM % 150000))
10
11     echo "[Test #$i] Ejecutando busqueda: -Likes > $RANDOM_LIKES" >>
       $OUTPUT_FILE
12     python Main.py Buscar -Likes "> $RANDOM_LIKES" >> $OUTPUT_FILE 2>&1
13     echo "-----" >>
       $OUTPUT_FILE
14 done
15
16 echo "Pruebas finalizadas. Revisa el archivo $OUTPUT_FILE"
```

Listing 1: Script de automatización de pruebas de estrés